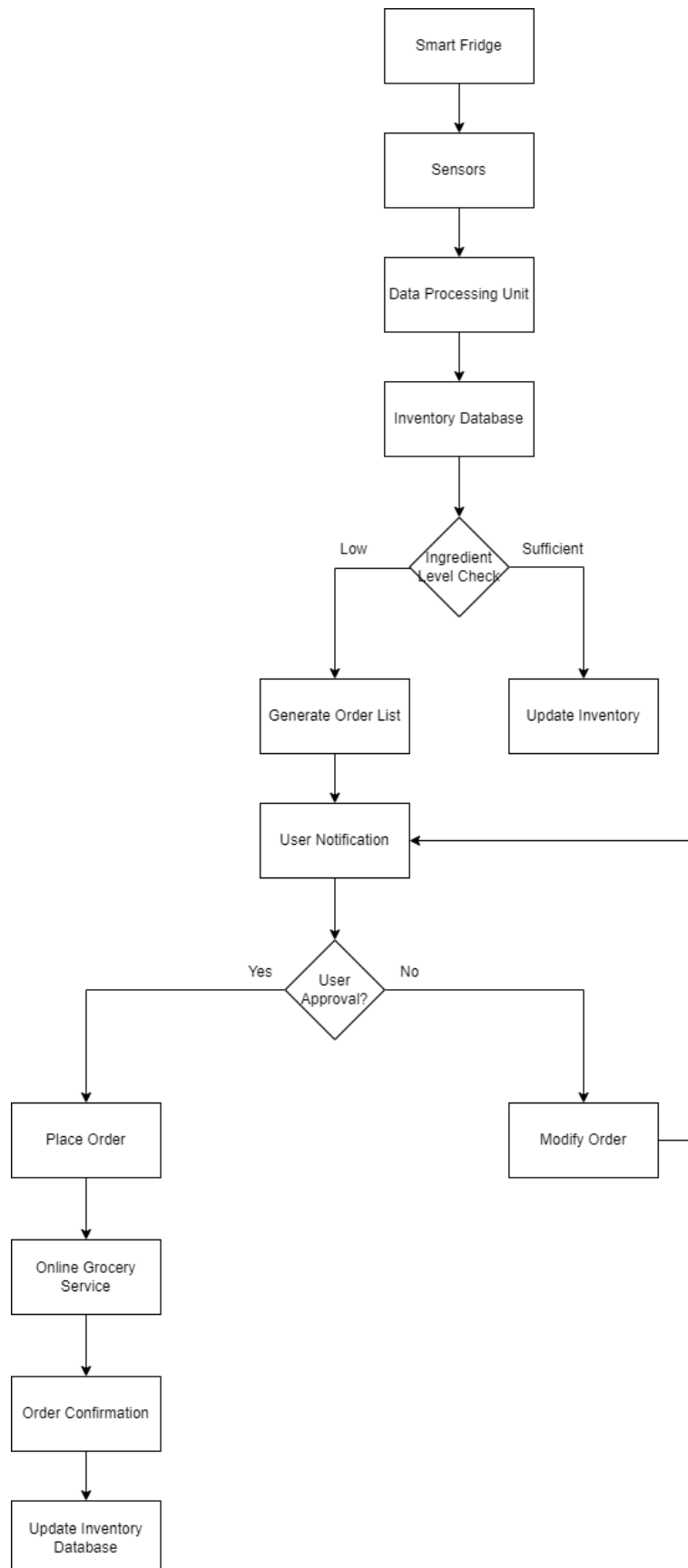
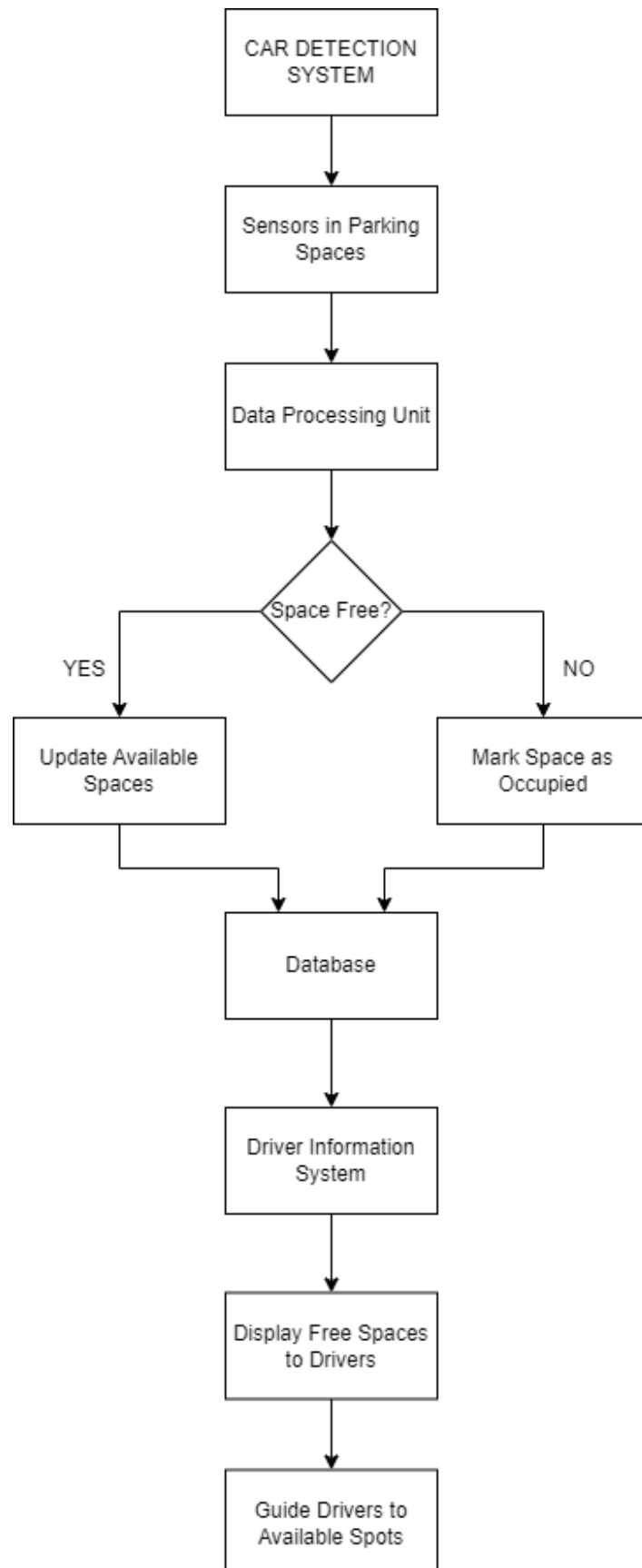






Step-by-Step Code Implementation

1. Install Required Libraries

Ensure you have the following libraries installed:

pip install opencv-python opencv-python-headless tensorflow numpy flask

2. Image Processing and Detection Code

Here's the complete code:

```
import os
import cv2
import numpy as np
import tensorflow as tf
from datetime import datetime
from threading import Thread
from flask import Flask, jsonify

# Path Configuration
CAPTURE_PATH = "captured_images"
MODEL_PATH = "fridge_model.tflite"

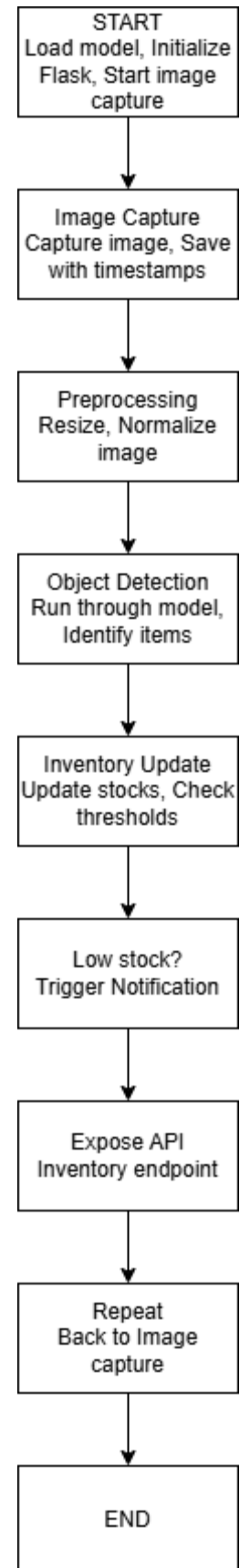
# Constants
IMAGE_INTERVAL = 10 # seconds
THRESHOLD_LEVELS = {'milk': 1, 'eggs': 6, 'vegetables': 2}

# Flask App for Inventory Notifications
app = Flask(__name__)
inventory = {}

# Load the TFLite Model
def load_model(model_path):
    interpreter = tf.lite.Interpreter(model_path=model_path)
    interpreter.allocate_tensors()
    return interpreter

interpreter = load_model(MODEL_PATH)
```

Flowchart



TSOGOO MUNKHZUL

Preprocessing

```
def preprocess_image(image):  
    """Resize, adjust brightness, and normalize the image."""  
  
    image_resized = cv2.resize(image, (224, 224))  
  
    image_normalized = image_resized / 255.0  
  
    return image_normalized.astype(np.float32)
```

Inference

```
def detect_items(image, interpreter):  
    """Detect items in the image using the TFLite model."""  
  
    input_details = interpreter.get_input_details()  
    output_details = interpreter.get_output_details()  
  
    # Preprocess and run inference  
  
    image_preprocessed = preprocess_image(image)  
  
    input_data = np.expand_dims(image_preprocessed, axis=0)  
    interpreter.set_tensor(input_details[0]['index'], input_data)  
    interpreter.invoke()  
  
    output_data = interpreter.get_tensor(output_details[0]['index'])  
  
    return parse_detection_output(output_data)
```

```
def parse_detection_output(output_data):  
    """Mock function to map model output to detected items."""  
  
    # Replace this with actual labels and logic for your model.  
    labels = ["milk", "eggs", "vegetables"]  
  
    detections = {}  
  
    for i, label in enumerate(labels):  
        detections[label] = output_data[0][i] # Example logic  
  
    return detections
```

Image Capture

```
def capture_images():
```

TSOGOO MUNKHZUL

```
"""Capture images at intervals or when triggered."""
```

```
if not os.path.exists(CAPTURE_PATH):
```

```
    os.makedirs(CAPTURE_PATH)
```

```
camera = cv2.VideoCapture(0)
```

```
while True:
```

```
    ret, frame = camera.read()
```

```
    if not ret:
```

```
        break
```

```
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
```

```
    filename = f"{CAPTURE_PATH}/image_{timestamp}.jpg"
```

```
    cv2.imwrite(filename, frame)
```

```
    print(f"Image captured: {filename}")
```

```
    process_image(filename)
```

```
    cv2.waitKey(IMAGE_INTERVAL * 1000)
```

```
camera.release()
```

```
cv2.destroyAllWindows()
```

```
def process_image(image_path):
```

```
    """Process captured images for detection."""
```

```
    image = cv2.imread(image_path)
```

```
    detections = detect_items(image, interpreter)
```

```
    update_inventory(detections)
```

```
# Inventory Management
```

```
def update_inventory(detections):
```

```
    """Update inventory based on detections."""
```

```
    global inventory
```

```
    for item, count in detections.items():
```

```
        inventory[item] = inventory.get(item, 0) + count
```

```
        if inventory[item] < THRESHOLD_LEVELS[item]:
```

```
            notify_low_stock(item)
```

TSOGOO MUNKHZUL

```
def notify_low_stock(item):  
    """Notify when stock is low."""  
    print(f"Low stock alert for {item}!")  
  
# Flask API for Notifications  
@app.route("/inventory", methods=["GET"])  
def get_inventory():  
    return jsonify(inventory)  
  
# Run Image Capture in a Separate Thread  
def start_image_capture():  
    capture_thread = Thread(target=capture_images, daemon=True)  
    capture_thread.start()  
  
if __name__ == "__main__":  
    start_image_capture()  
    app.run(host="0.0.0.0", port=5000)
```